

1.Introduction

1.1 Purpose

The purpose of this document is to provide a detailed and structured description of the requirements for the **AI Powered Complaint & Issue Resolution System**. This Software Requirements Specification (SRS) serves as a formal reference for developers, evaluators, testers, and other stakeholders to clearly understand the system's functionality, scope, operational behavior, and design constraints. It also acts as a baseline for system development, validation, and future enhancements by defining both functional and non-functional requirements in a precise and unambiguous manner.

1.2 Document Conventions

In this document, the term “User” denotes students or citizens who submit complaints, while “Admin” denotes authorized authorities responsible for managing and resolving complaints. This Software Requirements Specification (SRS) follows IEEE-830 / IEEE-29148 standards. All functional requirements are specified with priorities classified as **High, Medium, or Low**. The priority assigned to higher-level requirements is inherited by the related detailed requirements unless stated otherwise. Consistent formatting, font style, and section numbering are used throughout the document to ensure clarity and readability.

1.3 Intended Audience and Reading Suggestions

The intended audience for this SRS includes Project Evaluators / Faculty, Software Developers, System Administrators, and Future Contributors. Project evaluators and faculty members may use this document to assess the system design, scope, and implementation approach. Software developers and system administrators can refer to it for understanding functional requirements, system behavior, and maintenance aspects. Future contributors may use this SRS as a reference for system enhancements, upgrades, and long-term support.

1.4 Product Scope

The system is a web-based platform designed specifically for university environments, allowing students to register complaints and track their resolution status in a structured and transparent manner. The system utilizes Artificial Intelligence (AI) to automatically categorize complaints, assign appropriate priority levels, and assist university administrators in faster and more informed decision-making. The architecture of the system is scalable and can be extended in the future to support broader public service complaint management applications.

2. Overall Description

2.1 Product Perspective

The system is a standalone web application built using the MERN stack. It combines frontend, backend, and database functionalities with a separate AI microservice. This integration enables seamless communication between web components and machine learning features.

Component	Relationship
User (Student/Admin)	Interacts with the system through the React frontend
React Frontend (UI)	Captures user input and sends API requests to the backend
Rule-based AI (aiSmartFilter.js)	Currently classifies category & priority using keyword matching
Node.js + Express Backend	Handles APIs, business logic, authentication (JWT), and database operations
REST API Communication	Enables data exchange between Node.js backend and AI microservice

2.2 Product Functions

The major functions of Apna Wallet are:

- **User Authentication:** Secure login and registration using JWT for students and admins.
- **Complaint Submission:** Users can raise complaints related to hostel, internet, electricity, water, etc.
- **Automatic Classification:** The system automatically determines complaint category and priority using AI (currently rule-based, future ML-based).
- **Complaint Tracking:** Users can track complaint status (Pending → In Progress → Resolved) in real time.
- **Analytics Dashboard:** Provides insights such as complaint trends, category distribution, and resolution performance.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise	Priority
Students (Primary Users)	University students who raise and track complaints	Basic	High
Admin (University Staff)	Authorized personnel responsible for managing complaints	Medium	High
System (AI Module / Backend Services)	Automated components handling classification and processing	High	Medium

Operating Environment

- Frontend: Web browser (Chrome, Edge, Firefox)
- Backend: Node.js runtime
- AI Service: Python environment
- Database: MongoDB Atlas

2.4 Design and Implementation Constraints

- Technology Stack Constraint: The system must be developed using the MERN stack (React, Node.js, Express, MongoDB) with a separate AI microservice using Python and FastAPI.
- AI Processing Constraint: Currently uses rule-based NLP (keyword matching); future implementation will use ML models (TF-IDF + Logistic Regression).
- API Communication: All components must communicate via REST APIs between frontend, backend, and AI microservice.
- Authentication: Secure user authentication must be implemented using JWT.
- Data Storage: All complaint and user data must be stored securely in MongoDB.
- Scalability: The system should support modular microservice architecture for future AI upgrades.
- Performance Constraint: Complaint classification and API responses should be fast and efficient.

2.5 User Documentation

- Onboarding Guide: Simple walkthrough for new users to understand complaint submission and tracking.
- Help/FAQ Section: Provides answers to common issues and system usage.
- Dashboard Guidance: Clear UI instructions for viewing complaints and status updates.
- Admin Guide: Instructions for managing complaints, updating status, and viewing analytics.

2.6 Assumptions and Dependencies

- Users have access to the internet and a web browser.
- Users are registered and authenticated before accessing the system.
- The backend server and database are always available and operational.
- The AI module (rule-based or ML-based) is accessible via API.
-

2.7 User Interfaces

The UI follows modern responsive design principles using React.js and Tailwind CSS, supporting both light and dark themes. Key screens include:

- Home / Dashboard: Displays all complaints with their status (Pending, In Progress, Resolved) along with quick insights.

- **Raise Complaint Screen:** Allows users to submit complaints with automatic category and priority prediction.
- **Admin Panel:** Enables admins to view, filter, and update complaint statuses efficiently.
- **Analytics Dashboard:** Displays complaint trends, category distribution, and resolution metrics.

2.8 Hardware Interfaces

- **User Device:** System runs on standard devices such as laptops, desktops, or smartphones.
- **Display:** Supports responsive layouts across various screen sizes and resolutions.
- **Internet Connectivity:** Required for accessing the web application and API services.

2.9 Software Interfaces

- **Frontend:** React.js with Tailwind CSS for UI development.
- **Backend:** Node.js with Express.js for API handling and business logic.
- **Database:** MongoDB with Mongoose for data storage and schema management.
- **AI Service:** Python-based FastAPI service for complaint classification.
- **Libraries/Tools:** JWT for authentication, Axios/Fetch for API calls, Scikit-learn for ML model.

2.10 Communications Interfaces

Client-Server Communication: REST APIs over HTTP/HTTPS between frontend and backend.
Backend-AI Communication: REST API calls between Node.js backend and FastAPI microservice.
Data Format: JSON used for data exchange across all components.
Network Requirement: Stable internet connection required for real-time data interaction.

3. System Features : Complaint Submission and AI-Based Classification

3.1.1 Description and Priority

This feature enables registered users (students) to submit complaints through a web-based interface. The system processes the complaint description using Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to automatically predict the complaint category and priority level before storing it in the database.

This feature forms the core functionality of the system and ensures automation of complaint handling.

Priority Level: High
Implementation Cost: Medium
Risk Level: Medium

3.1.2 Stimulus / Response Sequences

1. User logs into the system using valid credentials.
2. User navigates to the “Raise Complaint” section.
3. User enters complaint title and description.
4. User clicks the “Submit” button.
5. The system validates the input fields.
6. The system sends the complaint text to the AI service.
7. The AI service processes the text and returns predicted category and priority.
8. The system displays predicted category and priority to the user.
9. The system stores complaint details in the database.
10. The system displays a success confirmation message.

In case of invalid input or service failure:

- The system shall display appropriate error messages.
- The system shall prevent incomplete submission.

3.1.3 Functional Requirements

REQ-1: The system shall allow registered users to submit complaints through a web interface.

REQ-2: The system shall validate mandatory input fields (title and description) before processing the complaint.

REQ-3: The system shall transmit complaint text securely to the AI classification service.

REQ-4: The system shall automatically assign a complaint category based on AI prediction.

REQ-5: The system shall automatically assign a complaint priority level based on AI prediction.

REQ-6: The system shall store complaint details including title, description, category, priority, user ID, and timestamp in the database.

REQ-7: The system shall display confirmation upon successful complaint submission.

REQ-8: The system shall display appropriate error messages if AI service is unavailable.

REQ-9: The system shall prevent submission of empty or invalid complaint data

System Feature: Complaint Tracking and Status Management

3.2.1 Description and Priority

This feature enables users to track the status of their submitted complaints and allows authorized administrators to update complaint lifecycle stages. The system visually represents complaint progress through predefined stages: Pending, In Progress, and Resolved.

This feature ensures transparency and structured issue resolution within the university environment.

Priority Level: High

Implementation Cost: Medium

Risk Level: Low

3.2.2 Stimulus / Response Sequences

1. User logs into the system.
2. User navigates to the dashboard.
3. The system retrieves the user's complaints from the database.
4. The system displays complaint details along with current status.
5. Administrator logs into the system.
6. Administrator selects a complaint.
7. Administrator updates complaint status (Pending → In Progress → Resolved).
8. The system saves the updated status in the database.
9. Updated status is reflected in the user dashboard.

In case of unauthorized access:

- The system shall deny status update requests.
- The system shall display an access restriction message.

3.2.3 Functional Requirements

REQ-10: The system shall allow users to view all their submitted complaints.

REQ-11: The system shall display the current status of each complaint.

REQ-12: The system shall allow authorized administrators to update complaint status.

REQ-13: The system shall visually represent complaint lifecycle stages.

REQ-14: The system shall restrict complaint status updates to authorized administrators only.

REQ-15: The system shall log status updates with timestamp and admin details.

REQ-16: The system shall display appropriate error messages for unauthorized access attempts.

System Feature: Admin Dashboard and Analytics

3.3.1 Description and Priority

This feature provides administrators with a centralized dashboard to monitor complaint statistics, analyze trends, filter complaints, and generate reports. The dashboard enhances decision-making efficiency through visual analytics.

Priority Level: Medium

Implementation Cost: Medium

Risk Level: Low

3.3.2 Stimulus / Response Sequences

1. Administrator logs into the system.
2. Administrator navigates to the dashboard.
3. The system retrieves complaint data from the database.
4. The system generates statistical summaries (Total, Pending, Resolved).
5. The system displays visual charts and analytics.
6. Administrator applies filters (Category, Priority, Status).
7. The system dynamically updates displayed results.
8. Administrator selects export option (CSV/PDF).
9. The system generates and downloads the requested report.

3.3.3 Functional Requirements

REQ-17: The system shall display total number of complaints.

REQ-18: The system shall display number of pending and resolved complaints.

REQ-19: The system shall provide graphical analytics using charts.

REQ-20: The system shall allow filtering of complaints by category, priority, and status.

REQ-21: The system shall allow administrators to export complaint data in CSV format.

REQ-22: The system shall allow administrators to export complaint data in PDF format.

REQ-23: The system shall update dashboard statistics dynamically when filters are applied

4. Non-Functional Requirements

4.1 Performance Requirements

The system shall meet the following performance criteria:

- The system shall respond to user requests within **2 seconds** under normal operating conditions.
- AI prediction response time shall not exceed **2 seconds per request**.
- The system shall support at least **100 concurrent users** without performance degradation.
- Database query response time shall not exceed **1 second** for standard CRUD operations.

4.2 Safety Requirements

Although the system does not involve physical hazards, the following logical safety requirements shall be enforced:

- The system shall prevent accidental deletion of complaint records.
- The system shall require confirmation before performing critical administrative actions.
- The system shall maintain audit logs for complaint updates to prevent misuse.
- The system shall ensure data integrity during storage and retrieval operations.

No external safety certifications are required for this software application.

4.3 Security Requirements

- The system shall use **JWT-based authentication** for user sessions.
- The system shall store passwords in encrypted (hashed) format.
- The system shall restrict admin functionality to authorized users only.
- The system shall protect API endpoints from unauthorized access.
- The system shall validate user inputs to prevent injection attacks.

4.4 Software Quality Attributes

Reliability

- The system shall maintain consistent behavior under normal operating conditions.
- AI classification accuracy shall be periodically evaluated and improved.

Usability

- The system shall provide an intuitive and responsive user interface.
- Error messages shall be clear and informative.

Maintainability

- The system shall follow a modular architecture separating frontend, backend, and AI services.
- Code shall follow standard programming conventions.

Scalability

- The system architecture shall allow horizontal scaling in future deployments.
- The system shall support potential integration with public service complaint systems.

Robustness

- The system shall handle invalid or incomplete user input gracefully.
- The AI service shall process noisy text inputs effectively.

Portability

- The system shall be deployable on standard cloud platforms or local servers without major configuration changes.

5. Project Plan

5.1 Team Members

The project is developed by the following team member:

- **Vansh Patil** – Project Developer

5.2 Division of Work

The overall project work is divided and carried out as follows:

- **Requirement Analysis:** Collection and analysis of functional and non-functional requirements.
- **System Design:** Architectural design, database design, and user interface design.
- **Frontend Development:** Implementation of user interfaces using React and Tailwind CSS.
- **Backend Development:** Development of server-side logic, REST APIs, and database integration using Node.js and Express.
- **AI/ML Development:** Design and training of NLP-based models for complaint category and priority predict.
- **Testing:** Unit testing, integration testing, and system testing.
- **Documentation:** Preparation of SRS and project-related documentation.

All responsibilities are handled by the project developer.

5.3 Time Schedule

Project Phase	Duration
Requirement Analysis	1 Week
System Design	1 Week
Development	3 Weeks
Testing	1 Week
Documentation	1 Week

Appendix A: Glossary

- **AI** – Artificial Intelligence
- **NLP** – Natural Language Processing
- **SRS** – Software Requirements Specification
- **JWT** – JSON Web Token
- **API** – Application Programming Interface
- **UI** – User Interface
- **RBAC** – Role-Based Access Control

Appendix B: To Be Determined (TBD) List

1. Cloud deployment strategy
2. Mobile application support
3. Advanced AI model enhancement

4. Continuous model retraining pipeline